Hardwired Control Unit Design

Lecture Notes — Week 6

PCC CS-401: Computer Organization and Architecture

August 6, 2026

Where We Left Off, and Where We're Going

Last week we built two hardware datapaths — a single-bus organization and a three-bus organization — and traced the same instruction, `R1` ← `R2` + `R3`, through each of them by hand. On the single-bus datapath the transfer took three control steps, T_1, T_2, T_3, using two hidden registers `Y` and `Z` to buffer the bus; on the three-bus datapath the same transfer completed in a single control step, at the cost of more wiring. Both traces, however, shared a gap we left open on purpose: *we* wrote T_1, T_2, T_3 ourselves, on paper. Nothing in the hardware we built last week decides "it is now T_2" or "assert Z_{in} this cycle." Something inside the CPU must make that decision automatically, for every instruction, every cycle — and identifying and building that something is this week's entire subject.

This lecture follows one thread through three questions. First, what does the control unit actually read, and what does it decide, every clock cycle — that is, what are its inputs and outputs? Second, what hardware generates the control-step sequence $T_1, T_2, \ldots$ automatically, instead of our writing it by hand? Third, given an instruction's opcode and the current step, how do we derive the actual logic gates that assert the right control signals? Throughout, we reuse the identical running example from Week 5, `R1` ← `R2` + `R3` on the single-bus datapath, and build the actual control unit that drives it, so that "the hardware decides" stops being a hand-wave and becomes gates and Boolean equations.

1 Control Unit Functions

1.1 A Question We Skipped

Return to Week 5's trace for a moment: register `Y` loaded from the bus at T_1, and register `Z` loaded from the ALU at T_2 — never the other way around. Who decided that ordering? Not the ALU, which only computes what it is told to compute; not the bus, which only carries whatever is driven onto it; not the register file, which only loads or drives a register when a signal tells it to. Every one of those components is *passive* — each moves data only when explicitly told to, and none of them "knows" what step of the instruction is currently executing. Something else has to watch the instruction and the clock, and tell every other component what to do, every single cycle. That something is the **control unit**.

1.2 Definition: Control Unit

The **control unit** is the hardware that, every clock cycle, examines the current instruction and the current control step, and **asserts the control signals** that make every other component —

registers, ALU, memory, buses — do the right thing for that instruction at that step. It is important to be precise about what the control unit does *not* do: it does not compute results itself, since that is the ALU's job; and it does not store data itself, since that is the job of the registers and memory. The control unit's entire responsibility is deciding *when* each of those components acts, and *which* of them acts at any given moment.

1.3 Control Unit: Inputs and Outputs

Before designing anything internal to the control unit, it helps to fix its interface — what goes in, and what comes out. Picture the control unit as a single black box, drawn as a tall central box, with its inputs entering from the left and its outputs leaving from the right, each output aligned on the same row as the input that most directly determines it. Four signals enter on the left: the opcode currently sitting in the instruction register (`IR`), the condition-code flags (Z, N, C, V) reporting the outcome of the last ALU operation, the current control step T_i, and the clock/reset lines that drive the control unit's own timing. Four categories of signal leave on the right, each row-aligned with the input that drives it: register gating signals ($X_{\text{in}}/X_{\text{out}}$ for every register X), the ALU's function-select lines, the memory read/write lines, and a next-step signal that tells the sequencing hardware to advance. This input/output contract — IR, flags, current step, and clock in; register gating, ALU select, memory control, and next-step out — is exactly the interface Hamacher's standard treatment of control-unit design specifies [1], and it is the contract every design choice in the rest of this lecture has to satisfy.

1.4 Notation: Register-Gating Control Signals

The control unit's most frequent job is telling registers when to load and when to drive the bus, so this lecture introduces precise notation for that: $X_{\text{in}} = 1$ means "gate register `X` to *load* whatever is currently on the bus, this cycle," and $X_{\text{out}} = 1$ means "gate register `X` to *drive* its contents *onto* the bus, this cycle." Applying this notation to Week 5's single-bus trace of `R1` ← `R2+R3` makes the connection concrete: at T_1, $R2_{\text{out}}$ and Y_{in} fire together, so `R2` drives the bus and `Y` latches whatever appears there; at T_2, $R3_{\text{out}}$ fires (putting `R3` on the bus), the ALU is set to add, and Z_{in} fires so the sum of `Y` and the bus value latches into `Z`; at T_3, Z_{out} and $R1_{\text{in}}$ fire together, so `Z` drives the bus and `R1` latches the final result. These three lines are exactly the control signals the control unit must generate automatically — everything from here forward is about building the hardware that asserts them without a human writing them down by hand.

1.5 Two Philosophies for Building a Control Unit

There are two fundamentally different ways to build the hardware that maps (opcode, step) pairs onto control signals. **Hardwired control** builds that mapping directly out of logic gates: fast, because the signals appear combinationally with no extra delay, but the wiring is fixed once the chip is built. **Microprogrammed control** instead stores the control signals for each step as "microinstructions" inside a small memory, and steps through them with a microprogram counter: slower per step, because reading a stored word costs a memory access, but far easier to change, because updating a microprogram means rewriting memory contents rather than rewiring gates. This week builds the classical, gate-level hardwired approach in full; Week 7 rebuilds the identical control unit as microprogrammed control, so the two can be compared directly on the same worked example.

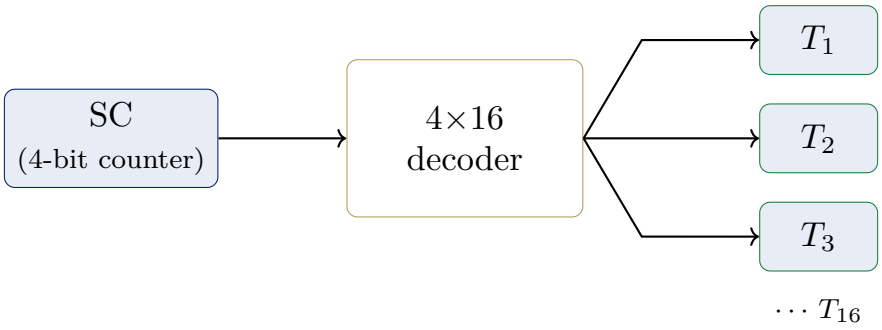
2 Generating the Control-Step Sequence

2.1 Recall: We Wrote T_1, T_2, T_3 by Hand

Returning to the trace once more — T_1: $Y \leftarrow [R2]$, with $[R2]$ on the bus, R2 driving the bus and Y latching it; T_2: $Z \leftarrow [Y] + [R3]$, with $[R3]$ on the bus, R3 driving the bus into the ALU, the ALU adding `Y`, and the result landing in `Z`; T_3: $R1 \leftarrow [Z]$, with $[Z]$ on the bus, `Z` driving the bus and `R1` latching it — notice that nothing in this table says *how* the hardware knows it is T_2 and not T_1 at any given moment. Some physical component inside the CPU has to keep track of "which step we are currently on," and identifying that component is the next step.

2.2 The Sequence Counter (`SC`)

The answer is a small counter register called the **sequence counter (`SC`)**, which holds the number of the current control step. A **step decoder** then converts `SC`'s binary value into one-hot signals $T_1, T_2, T_3, \ldots$, so that exactly one T_i line is active in any given cycle. Structurally, this is a two-stage pipeline of hardware: `SC` is a 4-bit counter register, feeding its binary count into a 4×16 step decoder, which in turn fans out into individual one-hot output lines — the decoder takes the counter's compact binary encoding and expands it into exactly one active line per step, which is precisely the encoding every other part of the control unit needs, since a signal like $Y_{\text{in}} = D_{\text{ADD}} \cdot T_1$ is far easier to wire from a one-hot T_1 line than from `SC`'s raw binary count.



cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-6, p.137 (the sequence-counter/timing-decoder portion of the control unit). This course numbers steps from T_1 (Mano's own text numbers from T_0), so a 4-bit/16-state counter yields $T_1, \ldots, T_{16}$ here rather than Mano's $T_0, \ldots, T_{15}$ [2].

This SC-plus-decoder pattern is the classical sequence-counter method for control-step generation.

2.3 How `SC` Advances

By default, every clock pulse simply increments the counter: $SC \leftarrow [SC] + 1$, which moves the step decoder's one-hot output to the next T_i automatically, with no external intervention needed. But at the *last* step of an instruction, the control unit instead asserts a special signal, SC_{clear}, which resets $SC \leftarrow 0$ and returns the decoder to T_1 — signaling that the current instruction is finished and it is time to begin fetching the next one. This reset is not optional bookkeeping: without SC_{clear}, the counter would simply keep counting upward forever, and no instruction would ever be recognized as complete. Applied to our running trace, SC_{clear} fires at T_3, since T_3 is the last step of `R1 ← R2+R3`.

2.4 Worked Trace Revisited: Where SC Points

Laying SC's actual numeric value alongside the RTL trace makes the mechanism concrete. At T_1, SC holds the value 0, the active line is $T_1 = 1$, and the RTL executed is $Y \leftarrow [R2]$. At T_2, SC holds 1, the active line is $T_2 = 1$, and the RTL is $Z \leftarrow [Y] + [R3]$. At T_3, SC holds 2, both $T_3 = 1$ and $SC_{\text{clear}} = 1$ are asserted, the RTL $R1 \leftarrow [Z]$ executes, and then SC resets to 0. This is exactly the piece Week 5 was missing: SC together with the step decoder *is* the hardware that turns "now it's T_2" from something we wrote by hand into something the circuit produces on its own, every cycle, with no human intervention.

2.5 Socratic Check: What Happens After T_3?

Suppose the designer forgot to wire up SC_{clear} entirely. What would the hardware do after T_3? Without the clear signal, SC would simply keep incrementing — T_4, T_5, and onward, forever. The step decoder would dutifully produce one-hot lines for these later steps, but since no instruction's state table defines any signals for T_4 or beyond, the datapath would simply idle: nothing would fire, no new instruction would ever be fetched, and the CPU would freeze after the first instruction completed its useful work. SC_{clear} is therefore not optional bookkeeping tacked onto the design — it is precisely what allows the control unit to cycle through a program's instructions at all, rather than freezing after the very first one.

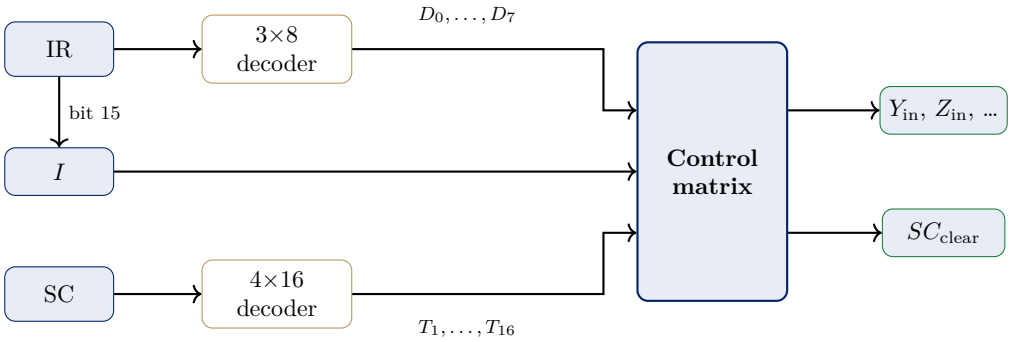
3 Hardwired Control Unit Design

3.1 The Missing Piece: Opcode Decoding

SC answers *when* — which T_i is active. But it says nothing about *which* instruction is running: Y_{in} should fire at T_1 for an ADD, while a LOAD instruction needs something entirely different at its own T_1. SC alone has no way to distinguish these cases. The missing piece is a new notational device: D_i, a one-hot **opcode decoder** output, where $D_{\text{ADD}} = 1$ exactly when the IR holds the ADD opcode, and every other $D_j = 0$ at that moment. With this notation in hand, the key structural fact of hardwired control emerges: every control signal will turn out to depend on *both* an opcode line (D_i) and a step line (T_j) together — never on either alone.

3.2 Two Decoders Feed the Control Matrix

Putting the two decoders side by side clarifies the overall shape of the design. The instruction register IR feeds a 3×8 operation decoder, which produces the one-hot $D_0, \ldots, D_7$ lines identifying which instruction is currently executing, and also loads the I flip-flop from IR bit 15 (the indirect-addressing mode bit). In parallel, the sequence counter SC feeds a 4×16 step decoder, producing the one-hot $T_1, \ldots, T_{16}$ lines identifying which control step is currently active. All three signals — the D_i lines, the I flip-flop, and the T_j lines — converge on a single combinational block called the **control matrix**, which produces the actual control signals as output: register-gating signals such as Y_{in} and Z_{in} on one output path, and SC_{clear} on another.



cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-6, p.137. `IR` bits 0–11 also feed the control logic gates directly (omitted above for clarity) [2].

The key claim this diagram makes precise is that every control signal is a Boolean function of the D_i lines, the I flip-flop, and the T_j lines together — the control matrix is nothing more than the circuitry that implements that Boolean function for every signal the datapath needs.

3.3 Building the State Table for `R1 ← R2+R3`

The bridge from the RTL trace we already know to the Boolean equations we are about to derive is a **state table**: a table listing, for each control step, which control signals are active. For `R1 ← R2+R3`, the state table has three rows. At T_1, the RTL $Y \leftarrow [R2]$ requires the signals $R2_{\text{out}}$ and Y_{in}. At T_2, the RTL $Z \leftarrow [Y] + [R3]$ requires $R3_{\text{out}}$, the ALU set to add, and Z_{in}. At T_3, the RTL $R1 \leftarrow [Z]$ requires Z_{out}, $R1_{\text{in}}$, and SC_{clear}. Every signal listed in the "active" column of this table is only ever asserted when *both* $D_{\text{ADD}} = 1$ *and* the listed $T_i = 1$ hold simultaneously — which is exactly the condition the Boolean equations below make explicit.

3.4 From State Table to Boolean Equations

Reading straight off the state table for `ADD` (using the opcode line D_{ADD}) produces one equation per control signal:

$$Y_{\text{in}} = D_{\text{ADD}} \cdot T_1, \qquad Z_{\text{in}} = D_{\text{ADD}} \cdot T_2, \qquad R1_{\text{in}} = D_{\text{ADD}} \cdot T_3, \qquad SC_{\text{clear}} = D_{\text{ADD}} \cdot T_3.$$

Each equation reads exactly the way it looks: "this signal fires when this instruction is running *and* we are at this step." Crucially, this is only half the story for any signal shared across multiple instructions — if a *different* instruction also needs, say, Z_{in} at one of its own steps, that instruction's term gets **OR**'d into the very same equation, which is exactly the situation the next worked example constructs.

3.5 Worked Example: Deriving Z_{in} by Hand

Suppose `SUB` also uses register `Z` to hold its result, at its own T_2 — a reasonable assumption, since `SUB` has an identical three-step shape to `ADD`, differing only in which ALU function is selected at T_2. Deriving the combined equation for Z_{in} proceeds in three steps. First, from `ADD`'s state table, Z_{in} fires at $D_{\text{ADD}} \cdot T_2$. Second, from `SUB`'s state table, Z_{in} also fires at $D_{\text{SUB}} \cdot T_2$. Third, since *any* instruction whose state table lists Z_{in} at T_2 contributes a term, the two terms combine by OR:

$$Z_{\text{in}} = (D_{\text{ADD}} \cdot T_2) + (D_{\text{SUB}} \cdot T_2) = T_2 \cdot (D_{\text{ADD}} + D_{\text{SUB}}).$$

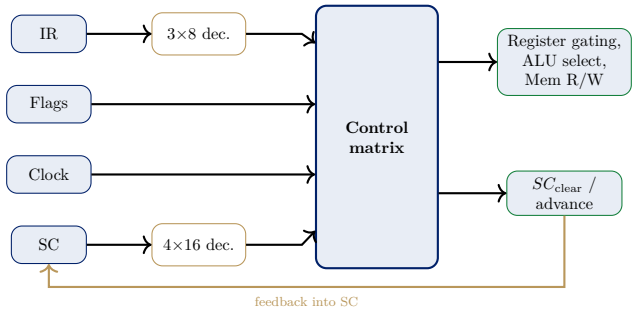
This worked derivation exposes the general pattern behind every control signal in a hardwired design: each signal's equation is an OR, taken across every instruction that uses it, of (opcode line · step line) terms.

3.6 The Control Matrix as Actual Logic Gates

The Boolean equation for Z_{in} derived above is not just algebra — it is literally a circuit diagram waiting to be drawn. Two AND gates realize the two product terms: the first AND gate takes D_{ADD} and T_2 as inputs and produces $D_{\text{ADD}} \cdot T_2$; the second AND gate takes D_{SUB} and T_2 as inputs and produces $D_{\text{SUB}} \cdot T_2$. Both AND-gate outputs then feed a single OR gate, whose output is the final Z_{in} signal. Two AND gates, one per instruction that needs Z_{in} at T_2, feeding one OR gate that combines them — this AND-OR structure is literally what the word "hardwired" means in hardwired control: the Boolean equation and the physical gate layout are the same object, viewed two ways.

3.7 Full Hardwired Control Unit Block Diagram

Assembling every piece built so far into one capstone diagram shows the complete hardwired control unit. On the input side, four sources feed the control matrix: the instruction register `IR` feeds a 3×8 decoder, which produces the D_i lines; the condition-code flags feed the matrix directly; the clock feeds the matrix directly; and the sequence counter `SC` feeds a 4×16 decoder, which produces the T_j lines. All four of these signal paths converge on the control matrix, drawn as one large combinational block at the center of the diagram.



cf. Mano, *Computer System Architecture*, 3rd ed., Fig. 5-6, p.137 (core: two decoders + SC + control logic gates), extended with explicit Flags/Clock inputs and the SC feedback loop [2, 1].

On the output side, the control matrix drives two categories of signal: register gating, ALU function select, and memory read/write signals, which together drive the datapath built in Week 5; and an SC_{clear}/advance signal, which feeds back — via an explicit feedback path drawn looping from the matrix's output back around to `SC`'s own input — to control whether `SC` advances normally or resets to zero. This feedback loop is what closes the system: the control matrix's own output determines whether the sequence counter that feeds the control matrix continues counting or restarts, cycle after cycle, for every instruction the CPU executes.

3.8 Socratic Check: Adding a New Instruction

Suppose a new `MUL` instruction is added to the instruction set, and it too uses register `Z` at its own T_2. What has to change in the hardwired control unit just built? Three things, all physical. First, the opcode decoder needs an entirely new output line, D_{MUL}, since the existing decoder has no way to signal "this is a `MUL` instruction." Second, every equation for a signal that `MUL` needs

— Z_{in} among them — must be **re-derived** from scratch to include the new term. Third, a new AND gate, computing $D_{\text{MUL}} \cdot T_2$, must be physically wired into the existing OR gate that already combines `ADD`'s and `SUB`'s contributions to Z_{in}. None of this is a software change: it is new gates, added to the actual chip, for every single signal the new instruction touches.

4 Trade-offs and Synthesis

4.1 Hardwired Control: Trade-offs

Hardwired control's advantages and costs both follow directly from the fact that it is pure combinational logic. On the positive side, control signals appear the *same cycle* they are needed, with no extra memory-access delay, and there is no wasted circuitry: the gates that exist are exactly the ones the instruction set architecture needs, nothing more. On the negative side, adding, removing, or fixing a bug in even one instruction means re-deriving Boolean equations and *physically rewiring* logic gates, exactly as the `MUL` example above demonstrated; and the control matrix as a whole grows with the number of instructions *times* the number of signals, so verifying it by hand becomes rapidly harder as the instruction set grows.

4.2 Where This Breaks Down

The worked example running through this lecture was deliberately small: one instruction (`ADD`), three steps, and a handful of signals, small enough that its equations fit on a single slide. A real instruction set architecture, by contrast, has dozens to hundreds of instructions, each contributing its own state table, and each contributing AND terms to *every* shared signal's OR equation. Every new instruction added to the ISA, and every microarchitectural bug fix, means re-deriving Boolean equations and re-wiring gates by hand — so design and verification cost explodes as the instruction set grows in complexity [3].

4.3 Bridge to Week 7

This scaling problem motivates a genuinely different idea: what if, instead of wiring Y_{in}, Z_{in}, $R1_{\text{in}}$, and every other signal directly into logic gates, we simply *wrote them down* — one row per control step, stored in a small memory — and let a counter step through the rows? Under this scheme, adding a new instruction becomes a matter of writing new rows into memory, not redesigning gates from scratch. The trade-off flips accordingly: each step becomes slightly slower, since reading a stored control word costs a memory access, but the design becomes far easier to build, verify, and modify. This is exactly **microprogrammed control**, and next week rebuilds this same `R1` ← `R2+R3` control unit using that approach.

4.4 The Two Threads Meet

Control-step count is not an independent design parameter — it depends directly on the bus organization chosen back in Week 5. On the single-bus datapath, three states (T_1, T_2, T_3) are needed, which means `SC` needs 2 bits, the step decoder has three outputs, and `ADD`'s state table has three rows. On the three-bus datapath, only one state (T_1) is needed, which means `SC` barely needs to exist at all, and `ADD`'s state table collapses to a single row. The consequence is direct: fewer control steps mean a smaller state table and fewer AND/OR gates in the control matrix — the hardware cost simply moves from control logic into bus wiring instead. Neither the datapath

choice made in Week 5 nor the control unit choice made this week is free; every design decision moves cost somewhere else on the chip, never eliminating it.

4.5 Summary

The **control unit** is the hardware that reads (opcode, step, flags) every cycle and asserts control signals for every other component to act on. The **sequence counter (`SC`)**, together with a step decoder, generates $T_1, T_2, \ldots$ automatically; SC_{clear} resets the counter back to T_1 at an instruction's last step. The **opcode decoder** produces one-hot D_i lines, and every control signal is an OR, taken across instructions, of $(D_i \cdot T_j)$ terms — the **state table** is what makes this systematic and mechanical rather than ad hoc. **Hardwired control** follows the pipeline state table $\rightarrow$ Boolean equations $\rightarrow$ AND/OR gates: fast, but rigid, since new instructions mean new gates, wired by hand. This rigidity is exactly what motivates **microprogrammed control**, taken up in Week 7, which stores control words in memory instead of wiring them into gates.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.

[2] M. Morris Mano. *Computer System Architecture*. Prentice-Hall, 3rd edition, 1993.

[3] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.